IES Horizonte

Manuel Soltero Díaz
CFGS Administración de Sistemas Informáticos en Red
2025/2026
IES Albarregas
13/03/2026

Introducción.....3
Explicación del código:..... 4
Ventana realizar_copia.....4
Ventana restaurar_copia.....8
Ventana ver_copias.....11
Ventana historial.....13
Ventana Principal..... 14
Scripts utilizados:..... 15

Manual del programador de la aplicación de Copias/restauración

Introducción

Este manual del programador describe en profundidad el funcionamiento interno de la aplicación de copias y restauración desarrollada en Python utilizando la biblioteca Tkinter para la interfaz gráfica. El objetivo principal del proyecto es facilitar la gestión de copias de seguridad de una base de datos MariaDB alojada en una máquina virtual Vagrant, permitiendo realizar copias, restaurarlas, visualizar los archivos generados y consultar un historial de operaciones.

La aplicación se organiza en una ventana principal desde la que se accede a las distintas funciones. Cada una de estas funciones se implementa mediante ventanas secundarias que recopilan datos del usuario y ejecutan comandos externos mediante el módulo subprocess. Estos comandos se envían a la máquina virtual, donde se ejecutan los scripts encargados de realizar las operaciones reales sobre la base de datos.

Además del código Python, la aplicación depende de dos scripts Bash externos, ubicados en la máquina virtual, que son fundamentales para el funcionamiento del sistema:

1. Script de copia (copia.sh)

Este script recibe cinco parámetros (usuario, contraseña, nombre de la base de datos, ruta de destino y nombre de la copia). Tras verificar que se han proporcionado correctamente, ejecuta un mysqldump comprimido con gzip para generar el archivo .sql.gz correspondiente.

Incluye también una tarea automática de mantenimiento que elimina copias con más de 30 días de antigüedad, garantizando que el almacenamiento no crezca de forma indefinida.

2. Script de restauración (restaurar.sh)

Este script también recibe cinco parámetros (usuario, nombre de la base de datos, ruta, contraseña y nombre de la copia). Tras validar los parámetros, descomprime el archivo .sql.gz seleccionado y lo restaura en la base de datos mediante mysql.

Su función es revertir la base de datos al estado guardado en la copia seleccionada.

Explicación del código:

El programa comienza importando los módulos necesarios para su funcionamiento. La instrucción **import tkinter as tk** carga la biblioteca Tkinter, utilizada para construir toda la interfaz gráfica de la aplicación. A continuación, **from tkinter import messagebox, ttk** incorpora componentes adicionales como las ventanas emergentes de aviso y los widgets avanzados del módulo *ttk*, entre ellos los desplegables. La línea **import subprocess** permite ejecutar comandos del sistema y comunicarse con la máquina virtual Vagrant donde se realizan las copias y restauraciones. Por último, **from datetime import datetime** se utiliza para obtener la fecha y hora actuales, necesarias para registrar cada operación en el historial.

```
import tkinter as tk
from tkinter import messagebox, ttk
import subprocess
from datetime import datetime
```

Ventana realizar_copia

La función **realizar_copia()** crea una ventana secundaria mediante **tk.Toplevel()** donde el usuario introduce los datos necesarios para generar una copia de seguridad. En la parte superior se muestra un título con **tk.Label(copia, text="Realizar Copia", font=("Arial", 16, "bold"))**. A continuación, se definen cinco variables **StringVar()** que almacenan el usuario, la contraseña, el nombre de la base de datos, la ruta destino y el nombre de la copia. Los campos correspondientes al nombre de la base de datos y a la ruta se configuran como solo lectura y se rellenan automáticamente con **var3.set("ies_horizonte")** y **var4.set("/home/vagrant/COPIAS")** para evitar modificaciones no autorizadas. Cuando el usuario pulsa el botón “Realizar copia”, se ejecuta la función interna **guardar_datos()**, que recoge los valores introducidos y lanza un comando remoto en la máquina virtual mediante **subprocess.run**, tal como se observa en la línea **comando_copia = f'vagrant ssh db1 -c "sudo bash /home/vagrant/SCRIPTS/copia.sh {usuario} {contraseña} {nombredb} {ruta} {nombrecopia}"'**. Si la copia se realiza correctamente, se ejecuta un segundo comando scp para transferir el archivo **.sql.gz** al equipo anfitrión y, finalmente, se registra la operación en el historial añadiendo una línea al archivo **historial.txt** junto con la fecha obtenida mediante **datetime.now().strftime(...).** En función del resultado de cada paso, la ventana muestra mensajes de éxito o error mediante **messagebox.showinfo** y **messagebox.showerror**.

```python
def realizar_copia():

    copia = tk.Toplevel()

    copia.title("Realizar Copia")

    copia.geometry("400x450")



    # Título

    tk.Label(copia, text="Realizar Copia", font=("Arial", 16,
"bold")).pack(pady=20)



    # Variables para guardar los datos

    var1 = tk.StringVar()

    var2 = tk.StringVar()

    var3 = tk.StringVar()

    var4 = tk.StringVar()

    var5 = tk.StringVar()



    # Campo 1

    tk.Label(copia, text="Usuario:").pack(pady=5)

    tk.Entry(copia, textvariable=var1, width=30).pack()



    # Campo 2

    tk.Label(copia, text="Contraseña:").pack(pady=5)

    tk.Entry(copia, textvariable=var2, width=30, show="*").pack()



    # Campo 3 (solo lectura)
```

```
    tk.Label(copia, text="Nombre Base de Datos:").pack(pady=5)

    entry_db = tk.Entry(copia, textvariable=var3, width=30,
state='readonly')

    entry_db.pack()

    var3.set("ies_horizonte")


    # Campo 4 (solo lectura)

    tk.Label(copia, text="Ruta destino:").pack(pady=5)

    entry_ruta = tk.Entry(copia, textvariable=var4, width=30,
state='readonly')

    entry_ruta.pack()

    var4.set("/home/vagrant/COPIAS")


    # Campo 5

    tk.Label(copia, text="Nombre copia:").pack(pady=5)

    tk.Entry(copia, textvariable=var5, width=30).pack()


    # Función para guardar el contenido de las variables

    def guardar_datos():

        usuario = var1.get()

        contraseña = var2.get()

        nombredb = var3.get()

        ruta = var4.get()

        nombrecopia = var5.get()


        try:
```

```
            # Primer comando: Realizar la copia

            comando_copia = f'vagrant ssh db1 -c "sudo bash
/home/vagrant/SCRIPTS/copia.sh {usuario} {contraseña} {nombredb} {ruta}
{nombrecopia}"'

            resultado1 = subprocess.run(comando_copia, shell=True,
capture_output=True, text=True, cwd="C:/Users/manue/Desktop/proyecto")



            if resultado1.returncode == 0:

                # Segundo comando: Copiar el archivo a Windows
(Anfitrión)

                scp = f'scp -P 2222 -i
C:/Users/manue/Desktop/proyecto/.vagrant/machines/db1/virtualbox/privat
e_key vagrant@127.0.0.1:{ruta}/{nombrecopia}.sql.gz
C:/Users/manue/CopiasBBDD/'

                resultado2 = subprocess.run(scp, shell=True,
capture_output=True, text=True, cwd="C:/Users/manue/Desktop/proyecto")



                if resultado2.returncode == 0:

                    # Obtener fecha actual

                    fecha = datetime.now().strftime("%d/%m/%Y
%H:%M:%S")



                    # Guardar en el historial

                    historial = f'vagrant ssh db1 -c "echo
\'{nombrecopia}.sql.gz - {fecha}\' >>
/home/vagrant/COPIAS/historial.txt"'

                    subprocess.run(historial, shell=True,
cwd="C:/Users/manue/Desktop/proyecto")
```

```
                    messagebox.showinfo("Éxito", "Copia realizada y
descargada correctamente")

                else:

                    messagebox.showerror("Error", "Copia realizada pero
error al descargar")

            else:

                messagebox.showerror("Error", "Error al realizar la
copia")

        except:

            messagebox.showerror("Error", "No se pudo ejecutar el
comando")



    # Botón

    tk.Button(copia, text="Realizar copia", command=guardar_datos,
width=20, bg="green", fg="white").pack(pady=20)
```

Ventana restaurar_copia

La función **restaurar_copia()** crea una ventana secundaria mediante **tk.Toplevel()** en la que el usuario puede restaurar una copia de seguridad previamente generada. La ventana muestra un título y cuatro variables **StringVar()** destinadas a almacenar el usuario, la contraseña, el nombre de la base de datos y la copia seleccionada. Igual que en la ventana de creación de copias, el campo del nombre de la base de datos se establece como solo lectura y se rellena automáticamente con **var3.set("ies_horizonte")** para evitar modificaciones no autorizadas. Para seleccionar la copia a restaurar, se utiliza un **ttk.Combobox** configurado como sólo lectura, cuyo contenido se carga dinámicamente ejecutando un comando remoto en la máquina virtual, tal como se observa en la línea **comando_listar = 'vagrant ssh db1 -c "find /home/vagrant/COPIAS -name \'*.sql.gz\' -type f -printf \'%f\\n\'"**. Una vez que el usuario introduce sus credenciales y elige una copia, el botón “Restaurar” ejecuta la función interna **ejecutar_restaurar()**, que primero valida la selección y luego construye el comando que invoca el script **restaurar.sh** en la máquina virtual, como se aprecia en **comando_restaurar = f'vagrant ssh db1 -c**

"sudo bash /home/vagrant/SCRIPTS/restaurar.sh {usuario} {nombredb} /home/vagrant/COPIAS {contraseña} {nombre_sin_extension}"'. Si la restauración se realiza correctamente, se registra la operación en el historial añadiendo una línea con la fecha obtenida mediante **datetime.now().strftime(...)**, y se muestra un mensaje de éxito; en caso contrario, se informa del error mediante **messagebox.showerror.**

```
def restaurar_copia():
    restaurar = tk.Toplevel()
    restaurar.title("Restaurar Copia")
    restaurar.geometry("400x450")

    # Título
    tk.Label(restaurar, text="Restaurar Copia", font=("Arial", 16,
"bold")).pack(pady=20)

    # Variables para guardar los datos
    var1 = tk.StringVar()
    var2 = tk.StringVar()
    var3 = tk.StringVar()
    var_copia = tk.StringVar()

    # Campo 1
    tk.Label(restaurar, text="Usuario:").pack(pady=5)
    tk.Entry(restaurar, textvariable=var1, width=30).pack()

    # Campo 2
    tk.Label(restaurar, text="Contraseña:").pack(pady=5)
    tk.Entry(restaurar, textvariable=var2, width=30, show="*").pack()

    # Campo 3 (solo lectura)
    tk.Label(restaurar, text="Nombre Base de Datos:").pack(pady=5)
    entry_db = tk.Entry(restaurar, textvariable=var3, width=30,
state='readonly')
    entry_db.pack()
    var3.set("ies_horizonte")

    # Lista de copias disponibles
    tk.Label(restaurar, text="Seleccionar Copia:").pack(pady=5)
    combo_copias = ttk.Combobox(restaurar, textvariable=var_copia,
width=27, state='readonly')
    combo_copias.pack()

   # Obtener lista de copias
```

```
    try:
        comando_listar = 'vagrant ssh db1 -c "find /home/vagrant/COPIAS
-name \'*.sql.gz\' -type f -printf \'%f\\n\'"'
        resultado = subprocess.run(comando_listar, shell=True,
capture_output=True, text=True, cwd="C:/Users/manue/Desktop/proyecto")

        if resultado.returncode == 0 and resultado.stdout.strip():
            copias = [linea.strip() for linea in
resultado.stdout.strip().split('\n') if linea.strip()]
            combo_copias['values'] = copias
        else:
            combo_copias['values'] = ["No hay copias disponibles"]
    except:
        combo_copias['values'] = ["Error al obtener copias"]

    # Función para restaurar
    def ejecutar_restaurar():
        usuario = var1.get()
        contraseña = var2.get()
        nombredb = var3.get()
        copia_seleccionada = var_copia.get()

        if not copia_seleccionada:
            messagebox.showwarning("Advertencia", "Por favor selecciona
una copia")
            return

        # Quitar la extensión .sql.gz del nombre
        nombre_sin_extension = copia_seleccionada.replace('.sql.gz',
'')

        try:
            # Comando para restaurar - orden: usuario, nombre_base,
ruta, password, nombre_copia
            comando_restaurar = f'vagrant ssh db1 -c "sudo bash
/home/vagrant/SCRIPTS/restaurar.sh {usuario} {nombredb}
/home/vagrant/COPIAS {contraseña} {nombre_sin_extension}"'
            resultado = subprocess.run(comando_restaurar, shell=True,
capture_output=True, text=True, cwd="C:/Users/manue/Desktop/proyecto")

            if resultado.returncode == 0:
                # Guardar en el historial
                fecha = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
```

```
                historial = f'vagrant ssh db1 -c "echo \'Restauración:
{copia_seleccionada} - {fecha}\' >>
/home/vagrant/COPIAS/historial.txt"'
                subprocess.run(historial, shell=True,
cwd="C:/Users/manue/Desktop/proyecto")

                messagebox.showinfo("Éxito", "Restauración realizada
correctamente")
            else:
                messagebox.showerror("Error", "Error al restaurar la
copia")
        except:
            messagebox.showerror("Error", "No se pudo ejecutar el
comando")

    # Botón
    tk.Button(restaurar, text="Restaurar", command=ejecutar_restaurar,
width=20, bg="blue", fg="white").pack(pady=20)
```

Ventana ver_copias

La función **ver_copias()** crea una ventana secundaria mediante **tk.Toplevel()** que permite visualizar todas las copias de seguridad almacenadas en la máquina virtual. La ventana muestra un título y, a continuación, construye un área de texto con barra de desplazamiento utilizando un **Frame**, un **Scrollbar** y un widget **Text**, tal como se observa en las líneas **scrollbar = tk.Scrollbar(frame) y texto = tk.Text(frame, yscrollcommand=scrollbar.set, width=60, height=15, font=("Arial", 12)).** Para obtener la lista de copias, la función ejecuta un comando remoto en Vagrant que lista los archivos **.sql.gz** dentro de la carpeta de copias, como se aprecia en **comando = 'vagrant ssh db1 -c "ls /home/vagrant/COPIAS/*.sql.gz 2>/dev/null | xargs -n 1 basename"'**. Si el comando devuelve resultados, estos se insertan en el área de texto línea por línea; si no existen copias o se produce un error, se muestra un mensaje informativo como “No hay copias disponibles” o “Error al listar las copias”. Finalmente, el widget **Text** se establece en modo solo lectura mediante **texto.config(state='disabled')** para evitar modificaciones por parte del usuario.

```
def ver_copias():
    ventana_copias = tk.Toplevel()
    ventana_copias.title("Ver Copias")
    ventana_copias.geometry("500x400")

    # Título
```

```
    tk.Label(ventana_copias, text="Copias Disponibles", font=("Arial",
16, "bold")).pack(pady=20)

    # Crear un área de texto con scroll
    frame = tk.Frame(ventana_copias)
    frame.pack(expand=True, fill='both', padx=20, pady=10)

    scrollbar = tk.Scrollbar(frame)
    scrollbar.pack(side='right', fill='y')

    texto = tk.Text(frame, yscrollcommand=scrollbar.set, width=60,
height=15, font=("Arial", 12))
    texto.pack(side='left', fill='both', expand=True)
    scrollbar.config(command=texto.yview)

    # Leer las copias desde Vagrant
    try:
        comando = 'vagrant ssh db1 -c "ls /home/vagrant/COPIAS/*.sql.gz
2>/dev/null | xargs -n 1 basename"'
        resultado = subprocess.run(comando, shell=True,
capture_output=True, text=True, cwd="C:/Users/manue/Desktop/proyecto")

        if resultado.returncode == 0 and resultado.stdout.strip():
            copias = resultado.stdout.strip().split()
            texto.insert('1.0', '\n'.join(copias))
        else:
            texto.insert('1.0', "No hay copias disponibles")
    except:
        texto.insert('1.0', "Error al listar las copias")

    texto.config(state='disabled')
```

Ventana historial

La función **historial()** crea una ventana secundaria mediante **tk.Toplevel()** que permite visualizar todas las operaciones registradas en el archivo **historial.txt** de la máquina virtual. La ventana muestra un título y construye un área de texto con barra de desplazamiento utilizando un **Frame**, un **Scrollbar** y un widget **Text**, como se observa en las líneas **scrollbar = tk.Scrollbar(frame) y texto = tk.Text(frame, yscrollcommand=scrollbar.set, width=70, height=15).** Para cargar el historial, la función ejecuta un comando remoto en Vagrant que lee directamente el archivo donde se almacenan tanto las copias realizadas como las restauraciones, tal como indica la instrucción **comando = 'vagrant ssh db1 -c "cat /home/vagrant/COPIAS/historial.txt"'**. Si el comando se ejecuta correctamente, el contenido completo del historial se inserta en el área de texto; si no existe o se produce un error, se muestra un mensaje como “No hay historial disponible” o “Error al leer el historial”. Finalmente, el widget **Text** se establece en modo solo lectura mediante **texto.config(state='disabled')** para impedir que el usuario modifique la información mostrada.

```
def historial():
    ventana_historial = tk.Toplevel()
    ventana_historial.title("Historial de Copias")
    ventana_historial.geometry("600x400")

    # Título
    tk.Label(ventana_historial, text="Historial de Copias",
font=("Arial", 16, "bold")).pack(pady=20)

    # Crear un área de texto con scroll
    frame = tk.Frame(ventana_historial)
    frame.pack(expand=True, fill='both', padx=20, pady=10)

    scrollbar = tk.Scrollbar(frame)
    scrollbar.pack(side='right', fill='y')

    texto = tk.Text(frame, yscrollcommand=scrollbar.set, width=70,
height=15)
    texto.pack(side='left', fill='both', expand=True)
    scrollbar.config(command=texto.yview)

    # Leer el historial desde Vagrant
    try:
        comando = 'vagrant ssh db1 -c "cat
/home/vagrant/COPIAS/historial.txt"'
```

```
        resultado = subprocess.run(comando, shell=True,
capture_output=True, text=True, cwd="C:/Users/manue/Desktop/proyecto")

        if resultado.returncode == 0:
            texto.insert('1.0', resultado.stdout)
        else:
            texto.insert('1.0', "No hay historial disponible")
    except:
        texto.insert('1.0', "Error al leer el historial")

    texto.config(state='disabled')  # Hacer el texto de solo lectura
```

Ventana Principal

La ventana principal de la aplicación se crea mediante **app = tk.Tk(),** donde se inicializa la interfaz gráfica y se configura su título y tamaño con **app.title("Ventana Principal") y app.geometry("400x350")**. En la parte superior se muestra una etiqueta con el texto “Copias / Restauración”, tal como se observa en **tk.Label(app, text="Copias / Restauración", font=("Arial", 16, "bold")).pack(pady=20)**. Debajo se colocan cuatro botones, cada uno asociado a una funcionalidad distinta: “Realizar copia”, “Restaurar copia”, “Ver copias” y “Historial”, enlazados respectivamente a las funciones **realizar_copia, restaurar_copia, ver_copias** y **historial**. Cada botón se configura con un tamaño fijo y colores diferenciados, como se aprecia en **tk.Button(app, text="Realizar copia", command=realizar_copia, width=30, height=2, bg="green", fg="white").** Al iniciarse la aplicación, se muestra un mensaje emergente recordando al usuario la importancia de realizar copias de seguridad mediante **messagebox.showinfo("Recordatorio", "Recuerda que tienes que hacer una copia de seguridad").** Finalmente, la instrucción **app.mainloop()** mantiene la ventana activa y gestionando eventos hasta que el usuario la cierre.

```
# Ventana principal
app = tk.Tk()
app.title("Ventana Principal")
app.geometry("400x350")

# Título
tk.Label(app, text="Copias / Restauración", font=("Arial", 16,
"bold")).pack(pady=20)
# Botones
```

```
tk.Button(app, text="Realizar copia", command=realizar_copia, width=30,
height=2, bg="green", fg="white").pack(pady=15)
tk.Button(app, text="Restaurar copia", command=restaurar_copia,
width=30, height=2, bg="blue", fg="white").pack(pady=15)
tk.Button(app, text="Ver copias", command=ver_copias, width=30,
height=2, bg="orange", fg="black").pack(pady=15)
tk.Button(app, text="Historial", command=historial, width=30,
height=2).pack(pady=15)

# Mensaje que es una ventana emergente al abrir la aplicación
messagebox.showinfo("Recordatorio", "Recuerda que tienes que hacer una
copia de seguridad")

app.mainloop()
```

Scripts utilizados:

Copia.sh

```
#!/bin/bash
# Manuel Soltero Díaz
# Script para hacer copias de la base de datos


#Comprobamos que ha introducido 5 parámetros
if [ "$#" -ne 5 ]; then
exit 1
    fi

#Asignamos variables a los parámetros
usuario=$1
password=$2
nombre_base=$3
ruta=$4
nombre_copia=$5



mysqldump -u "$usuario" -p"$password" "$nombre_base" | gzip >
"$ruta/$nombre_copia.sql.gz"
```

```
# Eliminamos copias de más de 30 días
find "$ruta" -name "*.sql.gz" -type f -mtime +30 -exec rm -f {} \;
```

Restaurar.sh

```
#!/bin/bash
# Manuel Soltero Díaz
# Script para hacer restaurar copias de la base de datos


#Comprobamos que ha introducido 5 parámetros
if [ "$#" -ne 5 ]; then
exit 1
    fi

#Asignamos variables a los parámetros

usuario=$1
nombre_base=$2
ruta=$3
password=$4
nombre_copia=$5


gunzip -c "$ruta/$nombre_copia.sql.gz" | mysql -u "$usuario"
-p"$password" "$nombre_base"
```